CO226-Apr2025 : Database Systems

Lab 04 : Writing SQL Queries

Suppose that you have started a new movie-rating website and you have been collecting data on reviewer's rating of various movies.

Figure below shows a certain instance of the populated database. Log into MySQL server and create a database named movieDB and create the three tables below.

```
mysql> SELECT * FROM MOVIE ORDER BY movie_id;
+----------+-------------------------+------+------------------+
| movie_id | title                   | year | director         |
+----------+-------------------------+------+------------------+
|      101 | Gone with the Wind      | 1939 | Victor Fleming   |
|      102 | Star Wars               | 1977 | George Lucas     |
|      103 | The Sound of Music      | 1965 | Robert Wise      |
|      104 | E.T.                    | 1982 | Steven Spielberg |
|      105 | Titanic                 | 1997 | James Cameron    |
|      106 | Snow White              | 1937 | NULL             |
|      107 | Avatar                  | 2009 | James Cameron    |
|      108 | Raiders of the Lost Ark | 1981 | Steven Spielberg |
+----------+-------------------------+------+------------------+
```

```
mysql> SELECT * FROM REVIEWER ORDER BY reviewer_id;
+-------------+------------------+
| reviewer_id | reviewer_name    |
+-------------+------------------+
|         201 | Sarah Martinez   |
|         202 | Daniel Lewis     |
|         203 | Brittany Harris  |
|         204 | Mike Anderson    |
|         205 | Chris Jackson    |
|         206 | Elizabeth Thomas |
|         207 | James Cameron    |
|         208 | Ashley White     |
+-------------+------------------+
```

```
mysql> SELECT * FROM RATING ORDER BY reviewer_id, movie_id, rating_date;
+-------------+----------+-------+-------------+
| reviewer_id | movie_id | stars | rating_date |
+-------------+----------+-------+-------------+
|         201 |      101 |     2 | 2011-01-22  |
|         202 |      106 |     4 | NULL        |
|         203 |      103 |     2 | 2011-01-20  |
|         203 |      108 |     4 | 2011-01-12  |
|         204 |      101 |     3 | 2011-01-09  |
|         205 |      103 |     3 | 2011-01-27  |
|         205 |      104 |     2 | 2011-01-22  |
|         205 |      108 |     4 | NULL        |
|         206 |      106 |     5 | 2011-01-19  |
|         206 |      107 |     3 | 2011-01-15  |
|         207 |      107 |     5 | 2011-01-20  |
|         208 |      104 |     3 | 2011-01-02  |
+-------------+----------+-------+-------------+
```

```sql
-- Create the database
CREATE DATABASE movieDB;
USE movieDB;

-- Create MOVIE table
CREATE TABLE MOVIE (
    movie_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    year INT NOT NULL,
    director VARCHAR(50)
);

-- Create REVIEWER table
CREATE TABLE REVIEWER (
    reviewer_id INT PRIMARY KEY,
    reviewer_name VARCHAR(50) NOT NULL
);

-- Create RATING table
CREATE TABLE RATING (
    reviewer_id INT,
    movie_id INT,
    stars INT CHECK (stars >= 1 AND stars <= 5),
    rating_date DATE,
    PRIMARY KEY (reviewer_id, movie_id),
    FOREIGN KEY (reviewer_id) REFERENCES REVIEWER(reviewer_id)
```

```
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (movie_id) REFERENCES MOVIE(movie_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Note how referential integrity options have been used in Operations like ON UPDATE and ON DELETE.

.sql file to create the database, tables and populate the tables is given with the lab sheet. Please refer to the script.sql file given in the zip folder.

Use that to create and populate the database and tables and then write the queries to complete the following tasks.

Lab Tasks

1. Select **movie id**s of each movie with its **title, reviewer id** and **stars** received.
2. Select **movie id**s of each movie with its **title, reviewer id** and **stars** received, where number of stars are less than or equal to three.
3. Select each **movie id** with its corresponding **title, reviewer id, reviewer name** and **stars** received.
4. Select each **movie id** with its corresponding **title, reviewer id, reviewer name** and **stars** received, where the number of **stars** received is equal to five.
5. Select movie **title** with its corresponding **reviewer name** and **stars**, where the movie's **rating date** is missing.
6. Select the details about the reviewers who have a last name called 'Martinez'.
7. Select the details about the ratings which have been rated before the 10th day of the month. Use substring comparison.
8. Show the effect of giving one more star to the movies reviewed by 'Brittany Harris'. Here select relevant details from the **RATING** table.
9. Select movie **titles** with its **reviewer name** and **stars** received. Order the result by movie title in the alphabetical order.
10. Select movie **titles** with its **stars** received and **rating date**. Order the result by **movie title** in the alphabetical order, then by **stars** and **rating date** both in descending order.

11. Write a nested query to list the details of the movies directed by a director
    A. who is also a reviewer.
    B. who is not a reviewer.
12. Write a nested query to list the details of the movie ratings,
    A. reviewed by the reviewer 'Sarah Martinez'.
    B. not reviewed by the reviewer 'Sarah Martinez'.
13. Find all the years that have a movie that received a rating of 4 or 5 and sort them in increasing order of the year. Write,
    A. a non-nested query.
    B. a nested query.
14. Some reviewers did not provide a date with their rating. Find the names of all reviewers who have a NULL value for the date. Write a nested query.
15. List the **movie titles** and average **ratings**, from the highest-rated to lowest-rated. If two or more movies have the same average rating, list them in alphabetical order.
16. Remove all ratings where the movie's year is before 1970 or after 2000
17. Remove all ratings where the rating date is NULL